

Upgrade from strimzi-0.51.0-kafka-4.1.0-trifork-1.10.0 to strimzi-1.0.1-kafka-4.1.0-trifork-2.0.0

This is the second and final hop of the two-step Strimzi migration from 0.48.0 to 1.0.1. **Do not start this guide until you have finished Upgrade Guide 1** (0.48.0 → 0.51.0). Your starting state should be:

- Strimzi Cluster Operator on 0.51.0.
- Kafka brokers running `ghcr.io/trifork/cheetah-infrastructure-utils-kafka:strimzi-0.51.0-kafka-4.1.0-trifork-1.10.0`.
- `kubectl get kafka -A` reports `Ready=True` on every Kafka CR.
- No unresolved Warning conditions on your Kafka CRs from guide 1.

Why two hops: Strimzi 1.0.0 removed the `v1beta2` CRD API entirely. Custom resources are still stored as `v1beta2` in etcd after guide 1, and they must be re-stored as `v1` before the 1.0.1 operator can read them. Strimzi 0.51 dual-serves `v1` and `v1beta2` and ships the `v1` API Conversion Tool that performs exactly this migration, which is why 0.51 is a good stop.

Reference: <https://strimzi.io/docs/operators/1.0.1/deploying.html#assembly-upgrade-str>

Steps for upgrading Strimzi Kafka

1. **Prerequisite - Kubernetes 1.30 or newer.** Unchanged from guide 1; Strimzi 1.0.1 has the same minimum.
2. **Prerequisite - Kubernetes API egress from Kafka pods.** If you added a NetworkPolicy in guide 1 to allow 443/TCP egress from broker/controller pods to the Kubernetes API for the `KubernetesSecretConfigProvider`, keep it.
3. **Move `spec.kafka.resources` to `KafkaNodePool1.spec.resources`.** In the `v1` API, broker resources are configured per node pool, not on the Kafka CR. `spec.kafka.resources` is deprecated in `v1beta2` and removed in `v1`.
 - For every node pool that runs brokers, add a `spec.resources` block with the values you had on `spec.kafka.resources`
 - Remove the `resources:` block from `spec.kafka:` on the Kafka CR.
4. **Verify no other "requires manual conversion" items apply to your CRs.** if you overlaid something custom, check the full list at <https://github.com/strimzi/strimzi-kafka-operator/blob/0.51.0/documentation/modules/upgrading/ref-kafka-v1-changes.adoc>. The most likely items to catch a customer overlay:
 - **Listener `authentication.type: oauth`** - not supported in `v1`. Migrate to `type: custom` with the equivalent `listenerConfig` fields (`oauthbearer.sasl.jaas.config` etc.).
 - **`authorization.type: keycloak` or `type: opa`** - not supported in `v1`. Migrate to `type: custom`.
 - **`KafkaUser.acls[].operation`** (singular) - must be renamed to `operations` (plural). Should already have been done as part of guide 1's prerequisites.

5. Convert all Strimzi CRs from **v1beta2** to **v1 apiVersion**. Two ways, pick one:

- **Option A - the official conversion tool** (`bin/v1-api-conversion.sh convert-file --file <path> --in-place`). Download from the [0.51.0 release](#). Docs: [con-api-conversion-v1.adoc](#) and [README.md](#). The tool takes a YAML file, walks every document in it, and rewrites Strimzi CRs to **v1**. It also applies "supporting automatic conversion" changes (renames `reconciliationIntervalSeconds` → `Ms`, deletes `spec.zookeeper`, etc.). Non-Strimzi documents are left alone.

Things to inspect before merging the tool's output:

- The tool re-serialises the entire YAML file, so expect noisy diffs unrelated to **v1beta2**→**v1**: list indentation flipped from 4-space to 2-space, map keys reordered, string quoting normalised, and **all comments stripped** (including operational annotations you may have added).
- The tool drops explicit values that equal a CRD default (for example `deleteClaim: false` on `KafkaNodePool` volumes). Runtime behaviour is unchanged, but the file loses documentation of intent.
- The tool cannot see CRs embedded as *strings* inside other YAML - e.g. patches in a Flux `Kustomization.spec.patches[*].patch` block. Those need a manual **apiVersion** bump regardless of which option you pick.
- **Option B - targeted manual apiVersion bump**. If you first verify (see "Cheetah note" below) that on your CRs the tool's only semantic change is **apiVersion**, just do a search-and-replace of `apiVersion: kafka.strimzi.io/v1beta2` → `apiVersion: kafka.strimzi.io/v1` across the same set of files. Preserves formatting and comments. **This is what Trifork chose** after inspecting the tool output.

Cheetah note. Before deciding between A and B, run the tool with `--in-place` on a scratch copy of your Strimzi CR files and compute a *semantic* diff against the originals. If the only difference is **apiVersion**, option B gives you the same result with a cleaner PR. Trifork's semantic-diff check on the test cluster reported exactly one changed path per file:

```
REMOVED: .apiVersion = 'kafka.strimzi.io/v1beta2'
ADDED:   .apiVersion = 'kafka.strimzi.io/v1'
```

Apply option A or B to every file containing a Strimzi CR - the `Kafka` and `KafkaNodePool` definitions, and all `KafkaTopic` and `KafkaUser` files, plus the embedded patches in your Flux `Kustomizations`. Once reconciled, verify: `kubectl get kafka,kafkanodepool,kafkatopic,kafkauser -A` should show everything Ready.

- (1.0.1-specific) Cross-namespace Entity Operator watches.** If any of your `Kafka` CRs sets `entityOperator.topicOperator.watchedNamespace` or `entityOperator.userOperator.watchedNamespace` to a namespace *different* from the Kafka CR's own namespace, you must add `STRIMZI_ENTITY_OPERATOR_WATCHED_NAMESPACE_ENABLED=true` to the Cluster Operator deployment's env before hopping to 1.0.1. Strimzi 1.0.1 flipped this feature gate's default from enabled to disabled.

In the HelmRelease values for the Cluster Operator add:

```
spec:
  values:
    extraEnvs:
      - name: STRIMZI_ENTITY_OPERATOR_WATCHED_NAMESPACE_ENABLED
        value: "true"
```

7. **Suspend Flux reconciliation for the Kafka operator Kustomization.** GitOps constraint: the tool in step 8 patches the CRDs in-cluster to flip `storage: true` from `v1beta2` to `v1`. If Flux keeps reconciling the checked-in 0.51 CRDs (which hardcode `storage: true` on `v1beta2`), it will revert the tool's changes within its next reconcile window. Suspend the operator's Flux `Kustomization` via git:

```
apiVersion: kustomize.toolkit.fluxcd.io/v1
kind: Kustomization
metadata:
  name: kafka-operator
  namespace: kafka
spec:
  suspend: true
```

Merge, then verify that the kustomization is suspended.

8. **Run `crd-upgrade` as a Kubernetes Job in-cluster.** The tool does three things in one shot: patches every CRD's `spec.versions[*].storage` to `v1`, re-encodes every stored Strimzi CR from `v1beta2` to `v1` bytes in etcd, and cleans each CRD's `.status.storedVersions` down to `[v1]`. All three are required before installing 1.0.1 CRDs.

The stock manifest in the [tool README](#) needs three additions to work in the typical `kafka` namespace:

- **PodSecurity `restricted` compliance** - the namespace enforces `restricted`, so the Job's pod needs an explicit `securityContext` at both pod and container level. Without it: `pods "strimzi-v1-api-conversion-xxxxx" is forbidden: violates PodSecurity "restricted:latest"`.
- **NetworkPolicy egress to the Kubernetes API on 443/TCP** - the `kafka` namespace has default-deny egress with per-workload exceptions. A generic Job pod won't match any existing allowance and will time out on its first API call with `HttpConnectTimeoutException`.
- **A distinct pod label** so the NetPol can select the Job's pod specifically.

Full manifest - save as `crd-upgrade-job.yaml`, substituting your API server IPs for `<API_CLUSTER_IP>` and `<API_ENDPOINT_IP>`. The job file is provided separately in the **"Reference Sheet"** file.

Apply through pipeline, wait, and inspect logs.

Expected log shape (three phases):

```

Checking that the CRDs are present and have the desired API versions.
Changing stored version in all Strimzi CRDs to v1:
  Updating KafkaTopic CRD
  ...
Updating all Strimzi CRs to be stored under v1:
  Updating KafkaTopic alertservice to be stored as v1
  ...
Changing stored version in statuses of all Strimzi CRDs to v1:
  Updating KafkaTopic CRD
  ...

```

Verify (powershell specific script used to check):

```

for c in kafkas kafkanodepools kafkatopics kafkausers; do
  kubectl get crd $c.kafka.strimzi.io -o jsonpath='{ "stored="}
{.status.storedVersions}{ " storage="}{.spec.versions[?
(@.storage==true)].name}{ "\n"}'
done
kubectl get crd strimzipodsets.core.strimzi.io -o jsonpath='{ "stored="}
{.status.storedVersions}{ " storage="}{.spec.versions[?
(@.storage==true)].name}{ "\n"}'

```

All should read `stored=["v1"] storage=v1`.

Do not delay between step 8 and step 9. As long as the 0.51.0 Cluster Operator is still running, its next reconcile will write a status update on some Strimzi CR - encoded as `v1beta2` - and silently re-add `v1beta2` to `.status.storedVersions`. If it should still fail, just re-apply the Job and continue.

Once verified, delete the Job resources.

9. **Bump CRDs, Cluster Operator chart, and broker image - in one coordinated PR pair.** so Flux applies the whole hop atomically when unsuspended.

- Remove `spec.suspend: true` from the operator's `Kustomization` (added in step 7).

10. **Clear stale `managedFields` on every Strimzi CR.** After the 1.0.1 CRDs land, and the kafka resources have rolled out, Flux Kustomizations that touch Kafka / KafkaNodePool / KafkaTopic / KafkaUser may fail with:

```
request to convert CR to an invalid group/version: kafka.strimzi.io/v1beta2
```

Root cause: every stored CR carries `managedFields[*]` entries pinned to the `apiVersion` that was current when they were written (`kafka.strimzi.io/v1beta2`). Server-side apply must interpret those entries against that `apiVersion` - which now doesn't exist - before it can compute a diff. Restarting Flux controllers doesn't fix this because the stale state lives on the *object itself*, not in any client cache.

Fix: patch each CR's `metadata.managedFields` to the special "clear" value `[{}]`. Kubernetes recognises this as "wipe all entries"; Flux and the Strimzi operators re-establish fresh entries at `v1` on their next reconcile. The resource's spec and status are untouched. As can be seen here: <https://kubernetes.io/docs/reference/using-api/server-side-apply/> under "clearing managedFields".

11. **Final verification.** (See Reference Sheet file attached) Check if:

- every CRD `served=[v1]` `storage=v1` `stored=["v1"]`
- Kafka CR `Ready=True` `op=1.0.1` `gen=X/X`
- no lines from the `managedFields` sweep, and all Flux Kustomizations `READY=True`.

Notes for upgrading

On the `authorizer.class.name` reserved-key note from guide 1 - still applies to 1.0.1. If you set `authorizer.class.name` in `spec.kafka.config`, remove it; the custom `CheetahKRaftAuthorizer` is registered by the Trifork image itself and the operator will warn (not fail) if you set it explicitly.

Client fallout during the roll. The 1.0.1 operator will roll every broker pod as part of the image bump. Expect the same class of client fallout you saw in guide 1: `librdkafka`-based clients (Go, .NET, Python, C++) load `ssl.ca.location` once at startup and cache the CA in memory. Clients that started before a rolling upgrade will pick up the new broker cert *only* on the next client-side restart. Redpanda-console falls into this category. If your OAuth listener uses a `brokerCertChainAndKey` cert issued by cert-manager and cert-manager happens to renew during the roll (as we observed once at Trifork), long-lived clients will need a `kubectl rollout restart` after the broker roll finishes.

Two footguns Trifork hit during the test-cluster upgrade - worth watching for.

1. **Timing race between `crd-upgrade` and the still-0.51 operator (step 8 → step 9 gap).** After `crd-upgrade` completes, the 0.51 Cluster Operator keeps running and reconciling - every status update it makes on a Strimzi CR is written back with `v1beta2` encoding, silently re-adding `v1beta2` to `.status.storedVersions`. Mitigation: run the `crd-upgrade` Job and merge the step-9 PRs back-to-back. If step 9 fails with a `storedVersions[...]: Invalid value: "v1beta2"` error on some CRD, just re-apply the Job (it's idempotent) and retry.
2. **Stale `managedFields` on stored CRs (step 10).** Even after all CRDs are correctly on `v1` and the operator has restarted, every existing CR still carries `managedFields[*]` entries pinned to `kafka.strimzi.io/v1beta2` - the `apiVersion` in force when Flux and the Strimzi operators first applied them. Kubernetes' server-side-apply must resolve each entry against its recorded `apiVersion`, which fails now that `v1beta2` is gone. Restarting Flux controllers doesn't help - this state is on the object, not in any cache. The bulk-patch in step 10 (`metadata.managedFields: [{}]`) is the fix.

Tests performed by Trifork

Documentation:

- Review release notes for 0.52.0, 1.0.0 and 1.0.1, and the two SECURITY-tagged advisories that shipped with 1.0.1.

- Check that no CRD field Cheetah relies on was removed between 0.51 and 1.0.1 (only field we care about here - none removed for Cheetah's shape).
- Cross-check `type: custom` listener/authz config against the 1.0.1 API reference.

Local test:

- Assess new and removed CVEs on the base image.
- Push the image to the container registry.
- Insert the image into the development infrastructure, run component tests, verify Redpanda-console reconnects, verify K2K jobs continue.
- Open a PR against the development infrastructure and confirm CI passes.

Cluster test:

- Run a job with a continuous input source.
- Ensure the job runs without issues.
- Perform the guide's steps in order (3 → 4 → 5 → 6 → 7 → 8 → 9 → 10 → 11). Steps 3, 5, 6, 7 and 9 each land through their own PR / GitOps commit - wait for reconciliation and a quick health check between each.
- Monitor broker rolls; expect ~1–2 minutes of client-side disconnection per broker as usual.
- After step 11, verify: no Warning conditions on the Kafka CR, `Ready=True`, `observedGeneration == generation`, `.status.operatorLastSuccessfulVersion == 1.0.1`, all node pools show the expected replica count, `kubectl get pods -n kafka -l strimzi.io/cluster=<name>` all Running 2/2.
- Restart any long-lived librdkafka-based clients (Redpanda-console, .NET consumers, etc.) if broker-cert rotation coincided with the roll.
- Verify no data loss on the continuous-input job.